강화학습 Day02 hw1 보고서

작성자: Doyeon Park

작성일: 2026.10.30

목차

1. 실험 환경 및 설정
2. 학습방법
3. 기본 환경에서의 학습 결과
4. 미끄러짐 유무 비교
5. 실행 방법 및 결과 파일
6. Codex·GPT 사용

1. 실험 환경 및 설정

항목	설정
Python	3.12.3
Gymnasium	1.4.0
NumPy	2.5.4
Matplotlib	3.11.2
환경	FrozenLake-v1, 기본 4×4 맵
미끄러짐	is_slippery=False
Q-table	16×4 배열, 초기값 0
학습 횟수	5,000 episodes
학습률 α	0.1
할인율 γ	0.95
탐험 확률 ε	max(0.05, 0.999 ** episode)
학습 행동 선택용 seed	0
평가 횟수	200 episodes
평가 방식	ε=0, Q 값 갱신 없음
평가 행동 선택용 seed	1000

FrozenLake 를 다음과 같이 MDP 로 정의하였다.

- **상태 S:** 현재 위치한 칸의 번호이며, 총 16 개이다.
- **행동 A:** 왼쪽 0, 아래 1, 오른쪽 2, 위 3 으로 총 4 개이다.
- **전이 P:** False 에서는 선택 방향으로 이동하고, True 에서는 선택 방향 또는 양옆 방향으로 각각 1/3 확률로 이동한다. 경계 밖으로 향하면 현재 칸에 머문다.
- **보상 R:** 목표에 도착하면 1, 일반 이동과 구멍 진입은 0 이다.
- **할인율 γ:** 0.95 를 사용하여 미래 보상을 할인한다.
-

현재 위치와 행동을 알면 다음 상태의 전이 확률을 설명할 수 있다. 미끄러짐이 있더라도 이 확률이 과거 이동 이력에 의존하지 않으므로 상태를 현재 칸 번호로 표현한다.

Return 은 현재부터 받게 될 보상을 할인하여 합한 값이다.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \cdots$$

예를 들어 세 번의 행동에서 보상을 순서대로 0, 0, 1 받았다면, γ=0.95 일 때 Return 은 다음과 같다.

$$G_t = 0 + 0.95 \times 0 + 0.95^2 \times 1 = 0.9025$$

Q 값은 특정 상태에서 특정 행동을 선택했을 때 얻을 것으로 기대하는 Return 의 추정치이다. 따라서 즉시 보상이 0 인 행동도 미래의 목표 도달에 기여하면 양의 Q 값을 가질 수 있다.

한편 코드의 returns 에는 할인율을 적용하지 않은 에피소드별 보상 합계를 저장하였다. 이렇게 하면 목표에 도착한 판이 1, 나머지가 0 이므로 성공 여부를 나타내는 기록으로도 사용할 수 있다.

2. 학습방법

행동 선택에는 ε-greedy 를 사용하였다. 확률 ε 로 전체 행동 중 하나를 무작위로 선택하고, 나머지 확률$(1-\epsilon)$로 현재 Q 값이 가장 큰 행동을 선택하였다. 최대 Q 값을 가진 행동이 여러 개이면 그중 하나를 무작위로 선택하였다.
탐험 확률은 처음에 1 이며, 학습이 진행됨에 따라 감소한다. 최소값을 0.05 로 설정하여 학습 후반에도 탐험을 계속하도록 한다.

행동 후 얻은 상태·행동·보상·다음 상태를 이용하여 다음 식으로 Q 값을 갱신하였다.

$$target = r + \gamma \, max_{a'} Q(s', a')$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha [target - Q(s,a)]$$

예를 들어 현재 Q 값이 0.2, 즉시 보상이 0, 다음 상태의 최대 Q 값이 0.8 이라면 다음과 같이 계산된다.

$$target = 0 + 0.95 \times 0.8 = 0.76$$

$$Q_{new} = 0.2 + 0.1(0.76 - 0.2) = 0.256$$

목표 도달이나 구멍 진입으로 실제 종료된 경우에는 이후 보상이 없으므로 목표값을 r 로 설정한다. 외부 시간 제한으로 중단된 경우에는 다음 상태의 가치를 포함하였다. 반복문은 종료 또는 중단 중 하나가 발생하면 끝내도록 구현하였다.

3. 기본 환경에서의 학습 결과

3.1 학습 그래프

미끄러짐이 없는 기본 환경에서의 학습 그래프에는 에피소드별 보상과 최근 100 판의 이동평균 성공률을 표시하였고, 학습이 진행되면서 목표에 도달하는 비율이 높아지는 경향을 볼 수 있었다.

학습 후반에도 실패가 남는 이유 중 하나는 ε 의 최소값을 0.05 로 유지하여 무작위 행동을 계속 수행하기 때문이라고 유추해볼 수 있다. (그림 1: non_slippery_results/training_results.png 별도 첨부)

3.2 Q-table 및 정책 분석

Q-table heatmap 은 16 개 상태와 4 개 행동의 Q 값을 숫자와 색으로 나타낸 것이다. 목표 직전인 14 번 상태에서 오른쪽으로 이동하는 행동은 즉시 보상 1 을 받으므로 Q 값이 약 1 로 학습되었다.

구멍과 목표 상태에서는 에피소드가 종료되므로 해당 상태에서 다음 행동을 선택하지 않는다. 따라서 이 상태들의 Q 값은 모두 0 이다. (그림 2: non_slippery_results/q_table_heatmap.png 별도 첨부)

정책 격자는 각 일반 상태에서 Q 값이 가장 큰 행동을 화살표로 표시하였다. 시작 상태 S, 구멍 H, 목표 G 이고, 최대 Q 값이 같은 행동이 여러 개이면 모두 표시하도록 구현하였다. 일반 칸의 Q 값이 모두 0 인 경우에는 방향을 임의로 표시하지 않고 별도로 표시하도록 하였다. (그림 3: non_slippery_results/policy_grid.png 별도 첨부)

1 번 상태에서는 오른쪽보다 왼쪽의 Q 값이 높게 나타났다. 왼쪽 Q 값은 약 0.735, 오른쪽은 약 0.453 이었다. 그러나 오른쪽으로 이동하면 목표까지 5 회, 왼쪽으로 이동하면 최단 7 회의 행동이 필요하므로, 최적으로 학습된 경우에는 오른쪽의 가치가 더 높아야 하지만 그렇지 않다. 이러한 이유로는 상태별 경험 부족또는 불완전한 가치 전파로 설명해볼 수 있다.

3.3 학습 후 평가

학습한 Q-table 을 q_table.npy 로 저장한 뒤 다시 불러와 평가하였다. 평가에서는 ε=0 으로 설정하고 Q 값을 수정하지 않았다. 별도 환경에서 총 200 회를 실행한 결과는 다음과 같다.

평가 항목	결과
평가 횟수	200 회
성공 횟수	200 회
성공률	100%
성공 경로 길이 평균	6 회
성공 경로 길이 최소	6 회
성공 경로 길이 최대	6 회

시작 위치에서 목표까지는 아래 방향과 오른쪽 방향으로 각각 세 칸의 이동이 필요하므로, 최소 6 회의 행동이 필요하다. 평가에서 모든 성공 경로의 길이가 6 회였으므로 시작 상태에서 최단 경로로 목표에 도착했음을 알 수 있다.

4. 미끄러짐 유무 비교

4.1 비교 조건 및 평가 결과

전이 불확실성의 영향을 확인하기 위해 is_slippery=False 와 True 에서 각각 Q-table 을 0 으로 초기화하여 학습하였다. 두 실험 모두 5,000 판, α=0.1, γ=0.95, 동일한 ε 감소식과 난수 시드를 사용하였다. 각 실험이 끝난 뒤 탐험과 Q 값 갱신 없이 별도 200 판을 평가하였다.

평가 항목	미끄러짐 없음 / 있음
성공 횟수	200/200 회 / 111/200 회
성공률	100% / 55.5%
성공 판 평균 행동 횟수	6 회 / 38.23 회
성공 판 최소 행동 횟수	6 회 / 6 회
성공 판 최대 행동 횟수	6 회 / 100 회

미끄러짐을 켠 경우 성공률은 44.5%p 낮아졌고, 성공한 판의 평균 행동 횟수는 6 회에서 38.23 회로 증가하였다. 여기서 평균·최소·최대 행동 횟수는 목표에 도달한 판만 대상으로 계산하며, 실패 또는 시간 제한으로 끝난 판은 제외한다.

미끄러짐이 없으면 선택한 경로를 그대로 이동할 수 있지만, 미끄러운 환경에서는 옆 방향으로 이동하거나 벽에서 제자리에 머물 수 있다. 평가의 ε 가 0 이어도 환경의 전이 무작위성은 남으므로, 같은 행동 정책을 사용하더라도 결과와 소요 행동 횟수가 달라질 수 있다.

4.2 학습 성공률의 변화

미끄러운 환경의 성공률 곡선은 전반적으로 상승하면서도 큰 변동을 보였다. 같은 상태·행동에서도 다른 다음 상태와 보상을 경험하므로 Q 업데이트 목표값이 달라질 수 있고, 학습 중에는 ε-greedy 탐험도 포함된다. 따라서 변동을 학습 실패로 단정하지 않고 전이 무작위성, 탐험, 현재 추정치의 변화를 함께 고려해야 한다.
(그림 4: slippery_results/training_results.png 별도 첨부)

4.3 정책과 Q 값의 변화

정책 격자의 화살표는 실제 이동 결과가 아니라 선택한 행동을 나타낸다. 미끄러운 환경에서는 벽이나 목표의 반대 방향을 선택하더라도 이동 후보 전체를 고려했을 때 유리한 행동이 될 수 있다.
(그림 5: slippery_results/policy_grid.png 별도 첨부)

목표 직전인 14 번 상태에서도 Q 값이 달라졌다. 미끄러짐이 없을 때 오른쪽 행동의 Q 값은 약 1 이지만, 미끄러짐이 있을 때는 약 0.588 이었다. 오른쪽을 선택해도 반드시 목표에 도달하지 않으며, 이후 보상도 할인되므로 즉시 목표 도달이 확실한 경우와 가치가 달라진다.

미끄러운 환경의 14 번 상태에서는 아래 행동의 Q 값이 약 0.738 로 가장 높았다.
(그림 6: slippery_results/q_table_heatmap.png 별도 첨부)

5. 실행 방법 및 결과 파일

Python 3.12 환경에서 가상환경을 만들고, 다음 명령으로 의존 패키지를 설치한다. pygame 은 시연 창을 표시하는 데 사용한다.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install gymnasium==1.4.0 numpy==2.5.4 matplotlib==3.11.2 pygame==2.6.1
```

코드가 있는 폴더에서 다음과 같이 실행한다. --no-demo 를 빼면 학습·평가와 그래프 저장 후 학습한 정책으로 한 판을 화면에 시연한다.

```
python "Day02_code.py"
python "Day02_code.py" --slippery

python "Day02_code.py" --no-demo
python "Day02_code.py" --slippery --no-demo
```

첫 번째 명령은 미끄러짐 없는 환경, 두 번째는 미끄러운 환경을 실행한다. 결과는 코드 파일 옆 non_slippery_results 와 slippery_results 폴더에 각각 저장한다. 각 폴더에는 q_table.npy, evaluation_results.json,

training_results.png, q_table_heatmap.png, policy_grid.png 가 생성된다. 제출용 result 폴더는 제출자가 미리실행한 결과를 넣어놓은 폴더로 별개이다.

6. Codex·GPT 사용

강의 자료의 예시 코드를 기반으로 실습을 진행하였다. Codex·GPT 는 다음 작업에 활용하였다.

- MDP, Return, Q 값, ε-greedy 및 학습·평가 코드 해석 및 설명
- Q-table heatmap, 정책 격자 구현
- 성공 경로 길이 기록, 평가 성공 경로 길이 기록과 결과 저장 기능 추가
- 미끄러짐 실험 옵션과 별도 결과 저장 기능 구현